



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών
Τομέας Ηλεκτρικών Βιομηχανικών Διατάξεων και
Συστημάτων Αποφάσεων

Προβλεπτική Παρακολούθηση Επιχειρηματικών Διαδικασιών με Παραγωγική Τεχνητή Νοημοσύνη

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Απόστολος Τζέλλας

Επιβλέπων: Γρηγόρης Μέντζας, Καθηγητής ΕΜΠ

Αθήνα, Ιούνιος 2026



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών
Τομέας Ηλεκτρικών Βιομηχανικών Διατάξεων και
Συστημάτων Αποφάσεων

Προβλεπτική Παρακολούθηση Επιχειρηματικών Διαδικασιών με Παραγωγική Τεχνητή Νοημοσύνη

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Απόστολος Τζέλλας

Επιβλέπων: Γρηγόρης Μέντζας, Καθηγητής ΕΜΠ

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή στις

Γρηγόρης
Μέντζας

Καθηγητής ΕΜΠ

Δημήτριος
Ασκούνης

Καθηγητής ΕΜΠ

Ευάγγελος
Σπηλιώτης

Επίκουρος Καθηγητής ΕΜΠ

Αθήνα, Ιούνιος 2026

Απόστολος Τζέλλας
Διπλωματούχος Ηλεκτρολόγος Μηχανικός και Μηχανικός Υπολογιστών

Copyright © Απόστολος Τζέλλας 2026
Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Απαγορεύεται η αναπαραγωγή, αποθήκευση και διανομή της παρούσας διπλωματικής εργασίας, εξ ολοκλήρου ή τμηματικά, για εμπορικό σκοπό. Επιτρέπεται η αναπαραγωγή, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για εμπορικό σκοπό πρέπει να απευθύνονται στον συγγραφέα.

Η έγκριση της διπλωματικής εργασίας από τη Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών του Εθνικού Μετσόβιου Πολυτεχνείου δεν υποδηλώνει αποδοχή των απόψεων του συγγραφέα.

Περίληψη

Η παρούσα διπλωματική εργασία εξετάζει τη χρήση μεθόδων Retrieval-Augmented Generation για το πρόβλημα της πρόβλεψης της επόμενης δραστηριότητας σε event logs επιχειρησιακών διαδικασιών. Το ακριβές κείμενο της περίληψης θα συμπληρωθεί αφού κλειδώσουν η μεθοδολογία, τα datasets και τα τελικά πειραματικά αποτελέσματα. Σύμφωνα με τον οδηγό της Βιβλιοθήκης, η ελληνική περίληψη πρέπει να παραμένει σύντομη, αυτοτελής και να μην υπερβαίνει τη μία σελίδα ή περίπου τις 500 λέξεις.

Λέξεις Κλειδιά: predictive process mining, retrieval-augmented generation, next-activity prediction, event logs, large language models

Abstract

This diploma thesis investigates the use of Retrieval-Augmented Generation for next-activity prediction in business process event logs. The final abstract will be refined once the methodology, datasets, and experimental findings are fully finalized. In line with the library guidelines, the English abstract is presented immediately after the Greek abstract and remains concise, self-contained, and limited to approximately one page.

Keywords: predictive process mining, retrieval-augmented generation, next-activity prediction, event logs, large language models

Acknowledgements

This page can include optional acknowledgements by the author. The final text should remain concise and preferably fit on a single page.

Contents

Περίληψη	5
Abstract	7
Acknowledgements	9
Εκτενής περίληψη	15
1 Αντικείμενο της εργασίας	15
2 Μεθοδολογική προσέγγιση	15
3 Κύρια ευρήματα και συμβολή	15
1 Introduction	17
1.1 Problem Context	17
1.2 Motivation	17
1.3 Objectives	17
1.4 Contributions	17
1.5 Thesis Structure	17
2 Related Work	19
2.1 Process Mining Foundations	19
2.2 Predictive Process Monitoring	19
2.3 Large Language Models	19
2.4 Retrieval-Augmented Generation	19
2.5 Embeddings and Reranking	19
2.6 Prompt Design for Prediction	19
3 Methodology	21
3.1 Data Preprocessing	22
3.2 Feature Construction and Selection	23
3.3 Split Strategy	25
3.3.1 Prefix Level	25
3.3.2 Trace Level	26
3.4 Retrieval Layer	27
3.5 Prompt Construction	27
3.6 Prediction Flow	28
4 Experiments	31
4.1 Experimental Setting	31
4.1.1 Datasets	31
4.1.2 Embedding Models	32
4.1.3 Reranker	32
4.1.4 Large Language Models	33
4.2 Evaluation Metrics	33
4.3 Experimental Factors	33
4.4 Reproducibility	33
5 Results	35
5.1 Overall Performance	35
5.2 Baseline versus Improved Variants	35
5.3 Effect of Retrieval and Prompting	35
5.4 Dataset-Specific Analysis	35

5.5	Early-Prefix Analysis	35
5.6	Discussion of Findings	35
6	Conclusion	37
6.1	Summary	37
6.2	Limitations	37
6.3	Future Work	37
	Bibliography	39

Εκτενής περίληψη

1 Αντικείμενο της εργασίας

Σε αυτή την ενότητα θα παρουσιαστεί συνοπτικά το ερευνητικό αντικείμενο της διπλωματικής, το πρόβλημα της πρόβλεψης επόμενης δραστηριότητας σε event logs, καθώς και το γενικό πλαίσιο του predictive process mining.

2 Μεθοδολογική προσέγγιση

Σε αυτή την ενότητα θα συνοψιστούν τα βασικά στάδια της προτεινόμενης μεθοδολογίας, από την προεπεξεργασία των δεδομένων και την κατασκευή prefixes μέχρι την ανάκτηση σχετικών παραδειγμάτων και τη χρήση μεγάλων γλωσσικών μοντέλων για την τελική πρόβλεψη.

3 Κύρια ευρήματα και συμβολή

Σε αυτή την ενότητα θα αποτυπωθούν τα βασικά πειραματικά ευρήματα, τα κύρια συμπεράσματα και η συνολική συμβολή της εργασίας σε σχέση με τη χρήση Retrieval-Augmented Generation στο predictive process mining.

Chapter 1

Introduction

1.1 Problem Context

This thesis studies next-activity prediction in business process event logs using predictive process mining and large language models. The practical goal is to estimate the most likely next step of an ongoing case from its observed prefix.

1.2 Motivation

Traditional predictive process monitoring methods rely on handcrafted representations or specialized sequence models. In contrast, Retrieval-Augmented Generation offers a way to support an LLM with semantically similar historical prefixes and traces, potentially improving prediction quality and interpretability.

1.3 Objectives

The main objectives of this thesis are to design, implement, and evaluate an LLM-based next-activity prediction pipeline with retrieval, and to study how retrieval, prompting, and experimental configuration choices affect performance.

1.4 Contributions

This section will summarize the concrete technical and experimental contributions of the thesis after the final evaluation setup is fixed.

1.5 Thesis Structure

The remainder of the thesis is organized into related work, methodology, experiments, results, and conclusion.

Chapter 2

Related Work

2.1 Process Mining Foundations

This section will introduce event logs, traces, events, prefixes, and the predictive process monitoring setting.

2.2 Predictive Process Monitoring

This section will review prior work on next-activity prediction and related predictive tasks in business processes.

2.3 Large Language Models

This section will discuss how LLMs can be used in structured prediction settings beyond open-ended text generation.

2.4 Retrieval-Augmented Generation

This section will summarize the RAG paradigm and why retrieved historical cases may help process prediction.

2.5 Embeddings and Reranking

This section will review dense retrieval, sentence embeddings, and optional reranking methods relevant to the thesis.

2.6 Prompt Design for Prediction

This section will discuss prompt design choices for classification-style or constrained prediction tasks.

Chapter 3

Methodology

This chapter describes the workflow followed to produce next-activity predictions, from the raw event log to the final evaluation stage. The pipeline includes:

- **Data preprocessing.** At this stage, raw XES event logs are parsed and converted into a structured format suitable for downstream processing. Cleaning and filtering steps are then applied so that only the events and attributes that are useful for prediction are retained.
- **Feature construction and selection.** Each trace is transformed into a collection of prefixes and corresponding target labels, where the label is the next activity to be predicted. In addition, selected information from recent events is preserved so that the model can exploit both control-flow and relevant attribute context.
- **Split strategy.** The generated examples are separated into retrieval and test collections according to the selected experimental protocol. Depending on the chosen mode, the split may be applied either at the prefix level or at the trace level before the final datasets are produced.
- **Retrieval of relevant historical examples.** For each query prefix, similar historical prefixes are retrieved from the training set using embeddings and vector search. The goal of this stage is to provide the language model with context from previously observed process executions.
- **Prompt construction for the language model.** The query prefix, the retrieved examples, and the required instructions are formatted into a unified prompt. In this way, the language model receives a structured input that guides it toward predicting the next activity.
- **Generation of the next-activity prediction.** The language model processes the final prompt and returns the predicted next activity for the examined prefix. This output is then isolated and converted into the final form used by the evaluation stage.
- **Performance evaluation.** The generated predictions are compared against the true next activities of the test set. Appropriate metrics are then used to assess the predictive quality of the system under different experimental settings.

Figure 3.1 summarizes the overall data flow of the proposed methodology, from raw event-log processing and split creation to retrieval, prompt assembly, next-activity prediction, and final evaluation. The diagram is intended as a high-level overview of the end-to-end workflow rather than as a one-to-one representation of every internal implementation step.

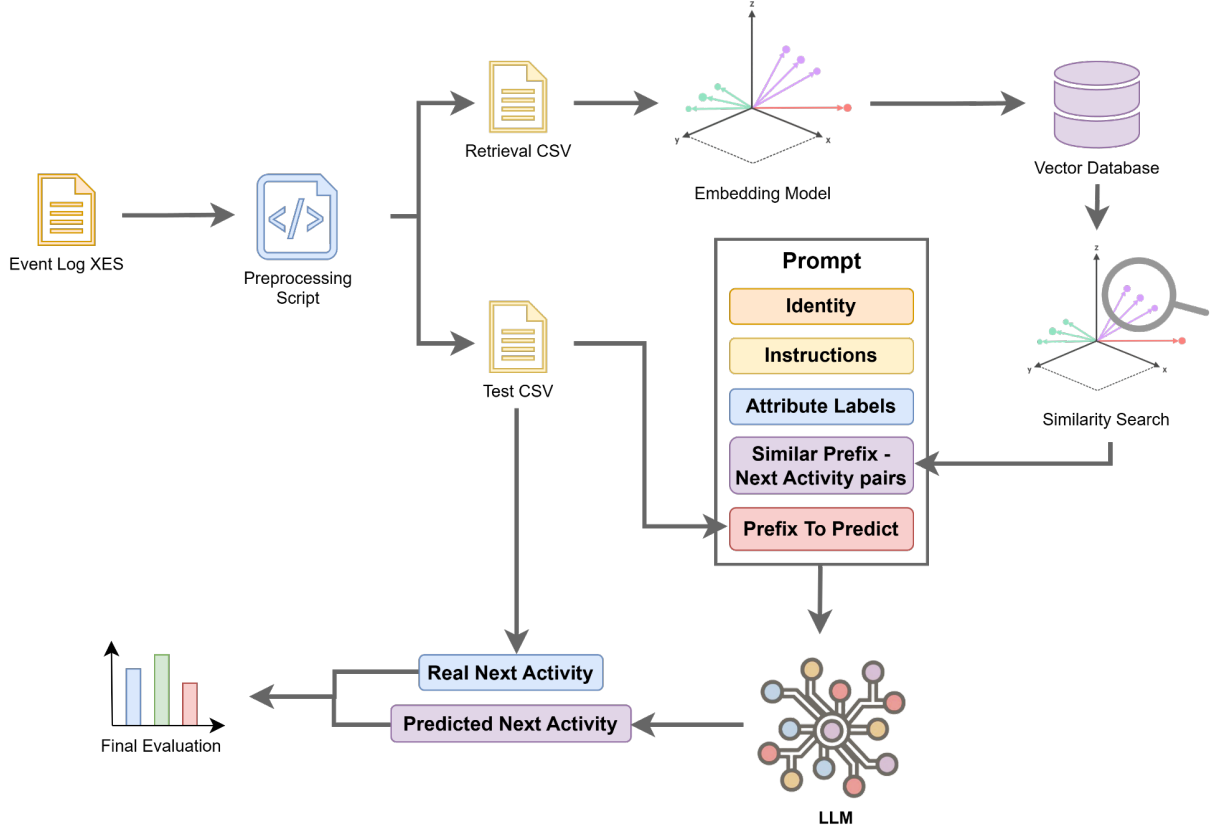


Figure 3.1: High-level overview of the full pipeline and evaluation flow

3.1 Data Preprocessing

Preprocessing starts from the original event log stored in XES format and converts it into a tabular representation in which each row corresponds to a single event of a process execution. At this stage, only the basic information required to reconstruct the event sequence of each case is preserved, namely the execution identifier, the activity name, the timestamp, and the available event-level attributes.

An initial cleaning step is then applied to remove records that are not suitable for the prediction setting. When lifecycle information is available in the log, only the events that indicate that an activity has been completed are retained. This ensures that each activity is represented once as an actual process step and prevents intermediate or auxiliary lifecycle transitions from distorting the execution sequence.

The next step restricts the set of attributes that are propagated to the later stages. Trace-level attributes that describe the execution as a whole are removed, with the exception of the trace identifier that is required to group events into the same trace. In contrast, event-level attributes that accompany each activity are preserved. This makes the dataset more consistent, avoids redundant information, and ensures that subsequent processing focuses mainly on the dynamic behavior of the process.

After cleaning, events are grouped by trace and arranged in the correct chronological order. This step is critical because next-activity prediction assumes an accurate representation of the path followed so far by each trace. For this reason, the sequence of events is reconstructed according to the available timestamps and, where necessary, the original appearance order is retained to resolve ties without altering the actual trace structure.

The outcome of this stage is not yet a learning-ready feature set, but a clean, consistent, and chronologically organized body of data that can support the subsequent construction of prefixes

and prediction targets.

Example before and after preprocessing

```
<event>
  <string key="lifecycle:transition" value="COMPLETE"/>
  <string key="concept:name" value="A_SUBMITTED"/>
  <date key="time:timestamp" value="2011-10-01T00:38:44.546+02:00"/>
</event>
<event>
  <string key="lifecycle:transition" value="COMPLETE"/>
  <string key="concept:name" value="A_PARTLYSUBMITTED"/>
  <date key="time:timestamp" value="2011-10-01T00:38:44.880+02:00"/>
</event>
<event>
  <string key="lifecycle:transition" value="SCHEDULE"/>
  <string key="concept:name" value="W_Completeren aanvraag"/>
  <date key="time:timestamp" value="2011-10-01T00:39:38.875+02:00"/>
</event>
<event>
  <string key="lifecycle:transition" value="START"/>
  <string key="concept:name" value="W_Completeren aanvraag"/>
  <date key="time:timestamp" value="2011-10-01T11:36:46.437+02:00"/>
</event>
<event>
  <string key="lifecycle:transition" value="COMPLETE"/>
  <string key="concept:name" value="A_ACCEPTED"/>
  <date key="time:timestamp" value="2011-10-01T11:42:43.308+02:00"/>
</event>
```

Listing 3.1: Raw trace fragment from the original XES log

Table 3.1: Dataframe after preprocessing

trace_id	activity	timestamp	lifecycle
173688	A_SUBMITTED	2011-10-01 00:38:44.546+02:00	COMPLETE
173688	A_PARTLYSUBMITTED	2011-10-01 00:38:44.880+02:00	COMPLETE
173688	A_PREACCEPTED	2011-10-01 00:39:37.906+02:00	COMPLETE
173688	A_ACCEPTED	2011-10-01 11:42:43.308+02:00	COMPLETE

The example illustrates an indicative trace fragment from the original XES log and its corresponding form after preprocessing. In the raw version, each event contains lifecycle information and timestamps, whereas after preprocessing only completed activities and the core attributes required by the later stages are retained.

3.2 Feature Construction and Selection

After preprocessing, each clean trace is transformed into a set of training examples suitable for next-activity prediction. The key idea is that a complete trace is not used as a single sample. Instead, it is decomposed into multiple prefixes. Each prefix represents the execution state of the process up to a specific point, while the target label is defined as the immediately following activity. In this way, the original event log is converted into a collection of prefix–target pairs that can be used both for retrieval and for final prediction.

Feature construction is based on two complementary sources of information. The first is the *control-flow*, namely the sequence of activities that has already been executed in the trace. The second is the *data payload*, which contains the values of the attributes associated with the individual events. In the proposed pipeline, these two aspects are not encoded as a conventional numerical feature vector. Instead, they are transformed into a structured textual representation, where the prefix preserves the sequential activity structure while also including selected attribute values from the most recent events.

Formally, let a trace be denoted by

$$\sigma = \langle e_1, e_2, \dots, e_n \rangle,$$

where e_i is the i -th event of the case. For any eligible position $t < n$, the corresponding prefix is

$$p_t = \langle e_1, e_2, \dots, e_t \rangle,$$

and the prediction target is the activity of the immediately following event, denoted by

$$y_t = a_{t+1}.$$

The generation of prefixes is controlled by the *base* parameter, which defines the minimum length from which examples start to be created. This avoids generating excessively short prefixes, which usually contain limited information and are not sufficiently representative of the actual state of the case. For each eligible point in a trace, the sequence of activities up to that point is used as the observed history, while the immediately following activity is used as the correct answer to be predicted.

The *gap* parameter determines the spacing between consecutively generated prefixes. Instead of creating a new example after every next event, the pipeline can construct prefixes at sparser intervals. This reduces the density of almost identical examples and controls the size of the resulting dataset without removing the essential ordering information. The use of gap-based prefix generation follows a design logic already discussed in predictive process monitoring literature [1]. In the final evaluation, this parameter is treated as one of the main experimental levers because it directly affects how finely the evolution of each trace is observed.

Taken together, *base* and *gap* define the set of eligible prefix lengths. If *base* = b and *gap* = g , these lengths can be written as

$$\ell_k = b + kg, \quad k = 0, 1, 2, \dots,$$

subject to $\ell_k < n$, where n is the length of the trace. The generated examples therefore take the form

$$(p_{\ell_k}, y_{\ell_k}).$$

This parameterization offers a simple way to control dataset granularity. Larger gap values reduce the number of generated examples and therefore help limit the size of the final dataset, but they also make the observation of the trace evolution coarser, since some intermediate states are no longer explicitly represented.

Particularly important is the m parameter, which defines how much recent attribute information is retained within the prefix. In the present implementation, the full activity sequence remains visible, but detailed *value blocks* are preserved only for the last m events. As a result, the model has access to the complete control-flow history, while only a limited and recent attribute context is kept in explicit form. This design is aligned with the literature on *last state encoding*, where the latest available snapshot of dynamic attributes is preserved and the method can be extended to *last m states* [2]. In this pipeline, the same idea is adapted into a textual representation by keeping only the last m value blocks within each prefix.

If a prefix ends at position t , the control-flow history remains fully visible, while explicit attribute-value blocks are retained only for the events whose indices belong to the set

$$\{i \mid \max(1, t - m + 1) \leq i \leq t\}.$$

Thus, the representation keeps the full sequential history but restricts the detailed data payload to the most recent m events.

These values are not stored as isolated independent fields. Instead, they form a snapshot of the most recently available attribute state up to the examined point. In other words, each new event updates the current attribute picture, and only the most recent relevant snapshots remain visible in the final representation. In addition, attribute names are compacted into shorter identifiers so that the prefix remains concise and usable in the later stages of the pipeline without losing its informational content.

Finally, deduplication is applied so that the same prefix–target pair is not stored multiple times. This reduces redundant information and leads to a more coherent collection of examples. The outcome of this stage is an organized set of textual prefixes that captures both the execution path of each trace and the most critical recent attribute context, as demonstrated in Listing 3.2.

```
event_A {Values},event_B {Values}
event_A,event_B,event_C {Values},event_D {Values},event_E {Values}
event_A,event_B,event_C,event_D,event_E,event_F {Values},event_G {Values},event_H {Values}
...
```

Listing 3.2: Indicative prefix representation

The above example shows successive prefixes following the same representation logic. For $base=1$, the first prefix is produced once the first two events have been observed. For $gap=3$, each subsequent prefix is extended by three new events. Finally, for $m=3$, detailed *Values* are retained only for the three most recent events, whereas older activities remain visible only as part of the *control-flow*.

3.3 Split Strategy

After feature construction, the pipeline can create the retrieval and test collections in two different ways. The difference lies in *when* the split is applied and in *which unit* is being separated. In both cases, deduplication follows the same internal key used by the implementation, namely the control-flow representation of the prefix together with its target label.

3.3.1 Prefix Level

In the first mode, the full event log is processed as a single body of data. All traces are first converted into one global table of prefix–target pairs. At this stage, global deduplication is already active, so repeated control-flow-and-label combinations are removed before the final split is created. As a result, the full prefix table contains only one retained copy of each unique training example under the implemented deduplication rule.

Once this globally deduplicated table has been produced, the split is applied directly at the prefix level. In the current implementation, 30% of the rows are sampled from the table to form the test set, while the remaining 70% form the retrieval set. In this configuration, the unit of separation is therefore the individual prefix row rather than the original trace. Since the split happens after all prefixes have already been generated, prefixes originating from the same original trace may end up on both sides, provided that they are not removed by the global deduplication step.

This mode is retrieval-oriented. It evaluates the system in a setting where the retrieval collection and the test collection are both derived from the same globally generated prefix space,

but exact duplicates according to the deduplication key are removed before row sampling takes place.

3.3.2 Trace Level

In the second mode, the split is applied before prefix generation. The original XES log is first divided into two valid XES files at the trace level: a training-side file and a test-side file. In the current implementation, 70% of the traces are assigned to the retrieval side and the remaining 30% to the test side. The traces are ordered chronologically according to their end timestamps, and the earlier portion is assigned to retrieval while the later portion is assigned to testing. In this way, the retrieval side is built from earlier completed cases and the test side from later completed cases.

After these split files are created, each side is processed through the same preprocessing and prefix-construction pipeline. At this point, the trace-level mode can be used in two variants: without cross-split deduplication and with cross-split deduplication. In the non-deduplicated variant, the training-side XES file is transformed into the retrieval CSV and the test-side XES file is transformed into the test CSV independently, without any shared deduplication state across the two sides. As a result, the same control-flow-and-label combination may appear in both retrieval and test if it is generated from different traces.

In the deduplicated variant, the same chronological trace split is kept, but a shared global deduplication state is used across retrieval and test during prefix generation. This means that once a control-flow-and-label combination has already been retained on the retrieval side, the same combination is not stored again on the test side. Because the retrieval side is processed first, the retained unique copy remains in retrieval and is excluded from the later test collection.

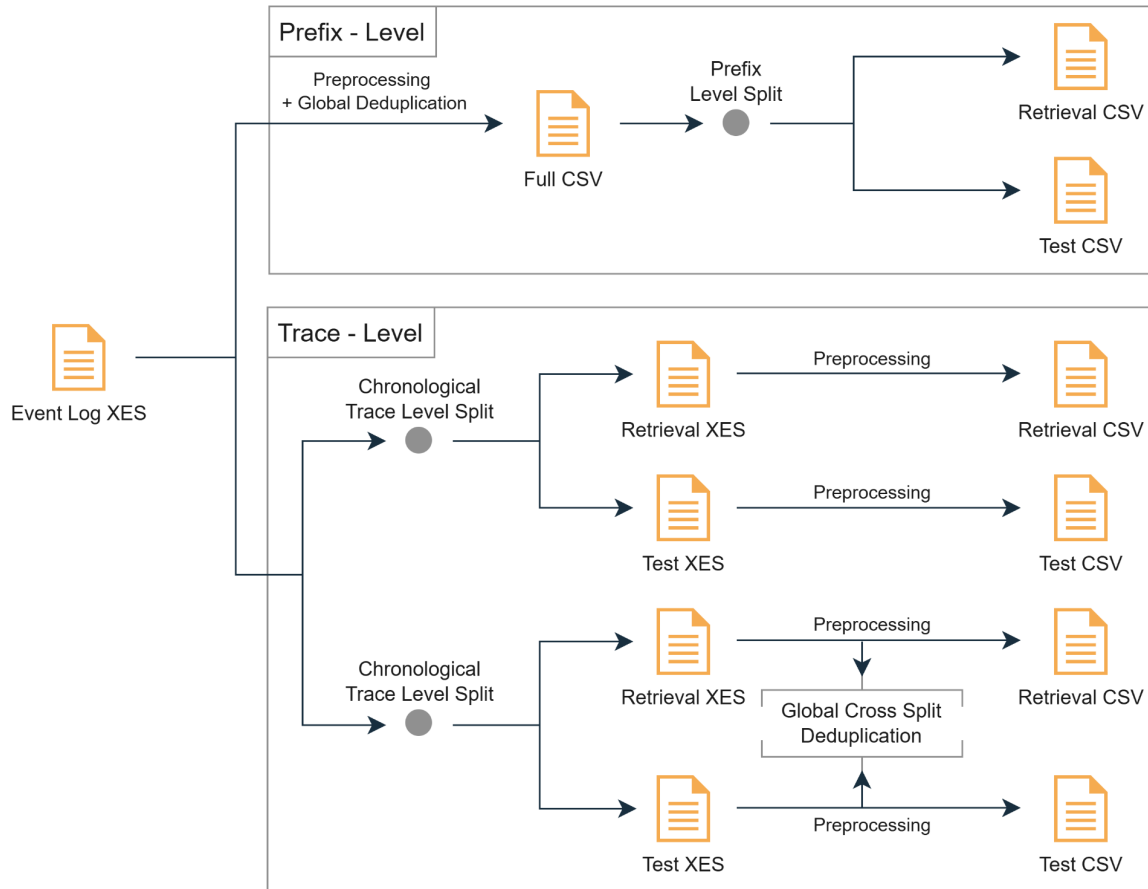


Figure 3.2: Split strategy pipeline

3.4 Retrieval Layer

Once the retrieval and test CSV files have been created, the next stage of the pipeline builds a searchable memory over the historical prefixes contained in the retrieval set. Each row of the retrieval CSV consists of a textual prefix and its corresponding next-activity label. The prefix is treated as the retrievable object, while the label is stored together with it so that, once a similar historical example is found, its known continuation can be passed to the prediction stage as contextual evidence.

To make this search possible, each retrieval prefix is transformed into a dense vector representation through an embedding model. In this way, prefixes that are similar in sequential structure and recent attribute context are mapped to nearby points in the vector space. After all retrieval prefixes have been encoded, their vector representations are stored together with the original textual prefix and the associated target label. As a result, the retrieval layer preserves both the compact numerical representation needed for similarity search and the original symbolic information needed for prompt construction.

During evaluation, each query prefix taken from the test CSV is encoded with the same embedding model. The resulting query vector is then compared against the stored retrieval vectors through similarity search, using a cosine-based comparison in the embedding space. The pipeline returns the top k most similar historical prefixes, where k is a configurable parameter of the experiment. Each retrieved item therefore contains three elements: the historical prefix itself, its known next-activity label, and its similarity score with respect to the query prefix. The role of this step is to provide a compact set of highly relevant past examples rather than the full retrieval collection.

An optional second ranking step can also be applied after the initial vector search. In this variant, a larger candidate pool is first collected by similarity search and then reordered according to a learned pairwise relevance criterion between the query prefix and each candidate prefix. The final top k examples are then selected from this reordered list. Whether this second ranking stage is used or not, the retrieval layer always produces the same type of output: a small ordered set of historical prefix-label examples that are passed to the prompt construction stage.

3.5 Prompt Construction

After the retrieval stage, the selected historical examples must be converted into a form that can be directly consumed by the language model. For this reason, the pipeline does not pass the query prefix alone. Instead, it builds a structured prompt that combines task instructions, contextual information, and the prefix whose next activity must be predicted.

The prompt is organized into a small number of clearly separated blocks, as shown in Figure 3.3. First, an *identity* block specifies that the model acts as a process-mining assistant specialized in predictive monitoring. Second, an *instruction* block defines the task itself, namely to predict the next activity immediately after the last activity of the given prefix and to return the answer in a strictly controlled format. Third, the prompt may include an *attribute legend*, which explains the compact attribute identifiers used in the textual prefix representation. Finally, the prompt includes the retrieved historical examples, each represented as a past prefix together with its known next activity.

After these contextual sections, the target prefix is appended as the final query to be solved by the model. In this way, the model receives both the current process state and a compact memory of similar historical continuations. The retrieved examples therefore function as in-context evidence, while the query prefix defines the specific prediction instance of interest.

Two prompt variants are used in the experiments. The first is a general retrieval-based prompt, which instructs the model to reason from the retrieved past traces and the values of

the most recent events. The second follows the same overall structure, but places stronger emphasis on the most recent attribute values retained by the *last-m* representation. The two variants therefore differ less in layout than in the weight they assign to the latest visible attribute context when predicting the continuation of the prefix.

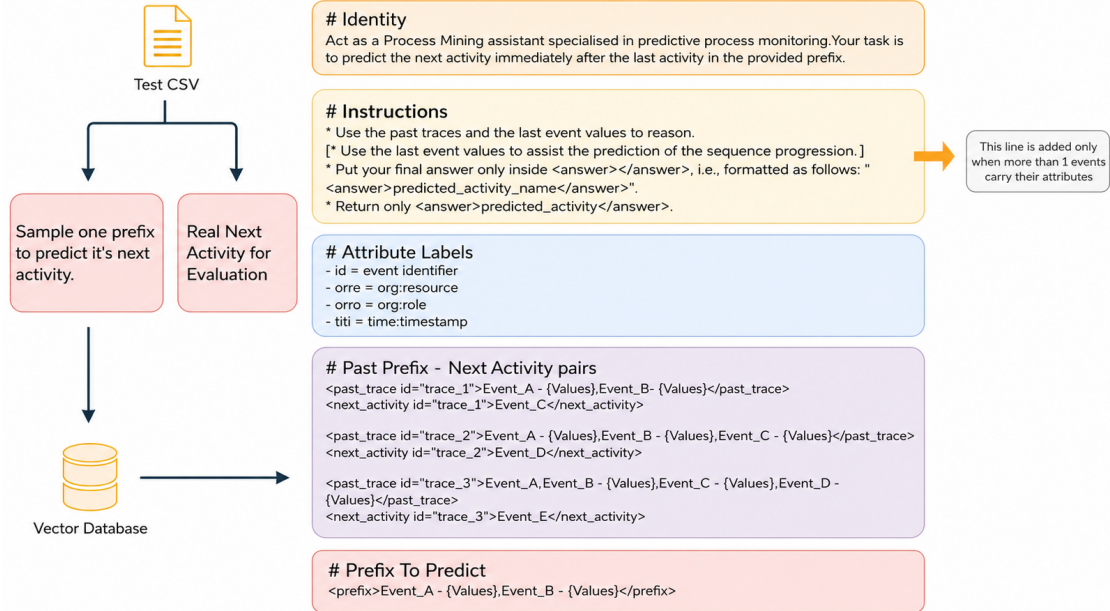


Figure 3.3: Vector database context within prompt construction

3.6 Prediction Flow

Once the final prompt has been assembled, it is submitted to the selected language model in order to generate the next-activity prediction for the query prefix. At this stage, the prompt already contains the task instructions, the retrieved historical examples, the attribute legend when available, and the prefix to be predicted. The role of the model is therefore not to reconstruct the context, but to infer the most plausible activity that should follow the current execution state.

To make the output interpretable and machine-readable, the model is asked to return its final answer in a strictly constrained form, namely inside `<answer></answer>` tags. In this way, the prediction stage does not depend on free-form generated text, but on an explicitly delimited output segment that can be isolated reliably even when the model produces additional reasoning or explanatory material. The generation itself is performed in a deterministic setting, so that the variability introduced by sampling is minimized during evaluation.

After the model response is received, the pipeline extracts the content enclosed within the `<answer></answer>` tags and treats this text as the predicted next activity. The extracted label is then normalized so that formatting differences do not affect the comparison with the ground truth. In particular, surrounding spaces and quotation marks are removed, and the label is converted to a standardized lowercase form before evaluation.

If the response does not contain a valid answer block, or if the extracted content corresponds to an empty or null-like output, the prediction is treated as invalid and is not counted as a successful evaluated case. Only responses that can be parsed into a valid activity label are retained as final predictions. For each accepted case, the normalized predicted label is stored together with the corresponding ground-truth next activity of the test row.

The final result of this stage is therefore a collection of prediction–target pairs, one for each valid query prefix processed during the evaluation run. These pairs form the direct input to the

metric computation stage, where correctness is assessed and aggregate performance values such as accuracy, macro-precision, macro-recall, and macro- F_1 are calculated.

Chapter 4

Experiments

4.1 Experimental Setting

This section summarizes the main components and configuration choices that define the experimental environment. It first presents the event logs used in the study and then outlines the embedding models, reranking component, and large language models considered in the evaluation.

4.1.1 Datasets

The evaluation uses four publicly available real-life event logs that are widely used in predictive process monitoring and process mining research. Together, they cover financial, administrative, and healthcare processes, and they differ substantially in size, control-flow variability, and attribute richness. This diversity is useful for assessing whether the proposed retrieval-augmented approach remains effective across logs with different behavioral structure and operational context.

The *BPI Challenge 2012* log captures the handling of personal loan and overdraft applications at a Dutch financial institution. It contains 262,200 events distributed across 13,087 traces. The process includes application registration, automated and manual checks, offer creation, offer acceptance, and final completion or cancellation, making it a strong benchmark for next-activity prediction under branching behavior and decision-intensive flows. In the local XES version used in this thesis, the log spans from October 1, 2011 to March 14, 2012, and it contains 24 distinct activity labels, or 36 activity classes when lifecycle transitions are taken into account [3].

The *BPI Challenge 2020 International Declarations* log contains travel-declaration and reimbursement cases related to international trips. It comprises 6,449 cases and 72,151 events. The process includes permit submission, approval steps by different organizational roles, declaration handling, and payment-related follow-up actions. Compared with the 2012 loan-application log, this dataset is smaller but still behaviorally rich, with repeated approval patterns, role-dependent routing, and multiple administrative checkpoints. In the local XES file used here, the observed event timestamps range from October 5, 2016 to May 10, 2020, and the log contains 34 distinct activity labels [4].

The *Sepsis Cases* event log captures the trajectories of patients treated for sepsis in a hospital environment. It contains 1,050 traces and 15,214 events covering 16 distinct activities, along with a relatively rich set of recorded attributes, including organizational groups, laboratory-test results, and checklist-related information. Because the cases reflect real clinical pathways, the log is less regular than strongly structured administrative datasets and offers a useful benchmark for testing how the proposed method behaves under noisy and medically heterogeneous process executions. This intuition is also supported by recent evidence showing that the Sepsis Cases log exhibits increased complexity and variability, especially for longer prefixes, which makes next-activity prediction more challenging [7]. In the local XES file used in this thesis, the timestamps range from November 7, 2013 to June 5, 2015 [6].

The *Hospital Billing* event log was extracted from the financial modules of a regional hospital ERP system. It contains 100,000 traces and 451,359 events, recording the activities required to bill bundled medical-service packages. The log does not describe the clinical treatment itself; instead, it focuses on the administrative billing workflow. This makes it particularly useful

Table 4.1: Summary of the real-world event logs used in the evaluation

Dataset	Time Span (months)	Traces	Events	Activity Labels
BPI Challenge 2012	5.4	13,087	262,200	24
BPI Challenge 2020 (International Declarations)	43.1	6,449	72,151	34
Sepsis Cases	18.9	1,050	15,214	16
Hospital Billing	37.2	100,000	451,359	18

for evaluating predictive methods in large-scale, repetitive back-office processes with moderate control-flow variation and substantial volume. In the present study, this dataset was not part of the baseline evaluation and was used mostly in the split-method evaluation. The local XES version spans from December 13, 2012 to January 19, 2016 and contains 18 distinct activity labels [5].

4.1.2 Embedding Models

In the retrieval component, two embedding models were examined. A MiniLM-based sentence-transformer model was selected as a reference point because it was also adopted in prior RAG-based work on next-activity prediction and yielded strong results there, making it an appropriate baseline for comparison [7]. In parallel, a more recent and explicitly retrieval-oriented BGE model was evaluated in order to assess whether a newer embedding family designed more directly for dense retrieval could offer additional benefits. This comparison made it possible to contrast a strong prior reference encoder with a more recent retrieval-focused alternative under a similar retrieval pipeline.

The MiniLM configuration uses `sentence-transformers/all-MiniLM-L12-v2` [9]. According to its model card, it maps sentences and short paragraphs to a 384-dimensional dense vector space and is intended for semantic search, clustering, and similarity-based retrieval, which makes it a reasonable lightweight baseline for prefix retrieval in process monitoring. Its underlying model family is rooted in MiniLM, which was designed through deep self-attention distillation to retain strong transformer behavior in a much smaller architecture [8]. In this thesis, MiniLM serves primarily as a compact baseline for testing whether a lighter encoder can still retrieve useful precedent prefixes.

By contrast, the BGE setting relies on `BAAI/bge-small-en-v1.5` [11]. This model belongs to the BGE family, which is motivated by recent work on embedding models optimized for retrieval-oriented use cases, including strong multilinguality, multiple retrieval functions, and improved granularity through self-knowledge distillation [10]. Although the exact deployed model in the code is the smaller English v1.5 variant, with a 384-dimensional representation space and a maximum input length of 512 tokens according to its model card, it follows the same broader design philosophy of the BGE line and is therefore a suitable retrieval-focused choice for the dense nearest-neighbor stage. In practice, it was selected as the main embedding model because it better reflects current retrieval-oriented embedding design while remaining lightweight enough for repeated experimentation.

4.1.3 Reranker

The reranking stage is a small extension of the retrieval pipeline. After the embedding model retrieves an initial pool of candidate prefixes from the vector database, a second model can be applied to reorder those candidates before the final top- k examples are passed to the predictor. In the implementation used here, this stage relies on `BAAI/bge-reranker-base`, which is loaded as a sequence-classification cross-encoder and scores query-candidate pairs directly rather than

comparing independent embeddings [13]. This design follows the standard reranking setting introduced in neural passage reranking with BERT-style models [12].

The main motivation for including this component was to test whether a more precise second-stage ranking step could improve the quality of the retrieved context with only a limited extension of the overall method. Since the reranker is applied only to a small candidate pool returned by dense retrieval, it preserves the general structure of the approach while offering a more targeted relevance assessment before prompt construction. In this way, reranking functions as a lightweight refinement of retrieval.

4.1.4 Large Language Models

The evaluation considered a mix of open-weight local models served through Ollama and proprietary models accessed through the OpenAI API. The goal was not to provide an exhaustive architectural analysis, but rather to compare models of different scale, design, and deployment setting under the same prediction task. Table 4.2 summarizes the main LLMs used in the experiments together with the model information reported by their corresponding providers [7, 14, 15, 16, 17, 18, 19, 20, 21].

Table 4.2: Summary of the LLMs considered in the evaluation

Model	Size	Architecture	Context	Embedding	Quant.
deepseek-r1:32b	32.8B	qwen2	131,072	5,120	Q4_K_M
gemma3:4b	4.3B	gemma3	131,072	2,560	Q4_K_M
gpt-oss:20b	20.9B	gptoss	131,072	2,880	MXFP4
mistral-small13.2:24b	24.0B	mistral3	131,072	5,120	Q4_K_M
phi4:14b	14.7B	phi3	16,384	5,120	Q4_K_M
qwen3:30b	30.5B	qwen3moe	262,144	2,048	Q4_K_M
gpt-4o-mini	N/P	Proprietary OpenAI	128,000	N/P	N/P
gpt-4.1-mini-2025-04-14	N/P	Proprietary OpenAI	1,047,576	N/P	N/P

The open-weight LLMs were served locally through Ollama (v0.30.10) on a workstation running Ubuntu 22.04.5 LTS. The platform featured an AMD Ryzen Threadripper PRO 5975WX processor with 32 cores, 251 GiB of system memory, and two NVIDIA GeForce RTX 4090 GPUs. Each GPU provided 24,564 MiB of VRAM, corresponding to approximately 24 GiB per device and about 48 GiB in total. These resources enabled the local execution of the quantized Ollama models used in the evaluation.

4.2 Evaluation Metrics

This section will describe accuracy, macro precision, macro recall, macro F1, and the earliness-oriented bucket analysis.

4.3 Experimental Factors

This section will summarize the controlled variables, such as top-k, gap, m, prompt variant, and reranking.

4.4 Reproducibility

This section will explain how runs, outputs, and result files are stored and compared.

Chapter 5

Results

5.1 Overall Performance

This section will present the main comparative results across configurations and datasets.

5.2 Baseline versus Improved Variants

This section will compare the baseline against later variants.

5.3 Effect of Retrieval and Prompting

This section will focus on retrieval depth, embedding choice, prompt variants, and reranking.

5.4 Dataset-Specific Analysis

This section will highlight differences in behavior across datasets.

5.5 Early-Prefix Analysis

This section will analyze model performance when only a small part of a trace is visible.

5.6 Discussion of Findings

This section will interpret the strongest and weakest empirical patterns observed in the experiments.

Chapter 6

Conclusion

6.1 Summary

This section will summarize the thesis objectives, methodology, and main findings.

6.2 Limitations

This section will discuss the main threats to validity and practical limitations of the current approach.

6.3 Future Work

This section will outline natural extensions, including stronger retrieval strategies, better earlyness metrics, and broader evaluation.

Bibliography

- [1] C. Di Francescomarino, M. Dumas, F. M. Maggi, and I. Teinemaa, “Clustering-Based Predictive Process Monitoring,” *IEEE Transactions on Services Computing*, vol. 12, no. 6, pp. 896–909, Nov.-Dec. 2019. [Online]. Available: <https://arxiv.org/abs/1506.01428>
- [2] I. Teinemaa, M. Dumas, M. La Rosa, and F. M. Maggi, “Outcome-Oriented Predictive Process Monitoring: Review and Benchmark,” *ACM Transactions on Knowledge Discovery from Data*, vol. 13, no. 2, pp. 1–57, 2019. [Online]. Available: <https://arxiv.org/abs/1707.06766>
- [3] B. van Dongen, “BPI Challenge 2012,” 4TU.ResearchData, 2012. [Online]. Available: https://data.4tu.nl/articles/dataset/BPI_Challenge_2012/12689204/1
- [4] B. van Dongen, “BPI Challenge 2020: International Declarations,” 4TU.ResearchData, 2020. [Online]. Available: <https://doi.org/10.4121/uuid:2bbf8f6a-fc50-48eb-aa9e-c4ea5ef7e8c5>
- [5] F. Mannhardt, “Hospital Billing - Event Log,” 4TU.ResearchData, 2017. [Online]. Available: <https://doi.org/10.4121/uuid:76c46b83-c930-4798-a1c9-4be94dfef741>
- [6] F. Mannhardt, “Sepsis Cases - Event Log,” 4TU.ResearchData, 2016. [Online]. Available: <https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460>
- [7] A. Casciani, A. Bernardi, E. Marrella, and F. M. Maggi, “Retrieval-Augmented Generation for next activity prediction in business process monitoring,” *Information Systems*, vol. 137, p. 102642, 2026. [Online]. Available: <https://doi.org/10.1016/j.is.2025.102642>
- [8] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” 2020. [Online]. Available: <https://arxiv.org/abs/2002.10957>
- [9] sentence-transformers, “all-MiniLM-L12-v2,” Hugging Face model card. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L12-v2>
- [10] S. Chen, Z. Liu, Y. Xiong, C. Feng, Z. Liu, and X. Wang, “M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.03216>
- [11] BAAI, “bge-small-en-v1.5,” Hugging Face model card. [Online]. Available: <https://huggingface.co/BAAI/bge-small-en-v1.5>
- [12] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” 2019. [Online]. Available: <https://arxiv.org/abs/1901.04085>
- [13] BAAI, “bge-reranker-base,” Hugging Face model card. [Online]. Available: <https://huggingface.co/BAAI/bge-reranker-base>
- [14] Ollama, “deepseek-r1:32b,” Ollama model library. [Online]. Available: <https://ollama.com/library/deepseek-r1:32b>
- [15] Ollama, “gemma3:4b,” Ollama model library. [Online]. Available: <https://ollama.com/library/gemma3:4b>
- [16] Ollama, “gpt-oss:20b,” Ollama model library. [Online]. Available: <https://ollama.com/library/gpt-oss:20b>
- [17] Ollama, “mistral-small3.2:24b,” Ollama model library. [Online]. Available: <https://ollama.com/library/mistral-small3.2:24b>
- [18] Ollama, “phi4:14b,” Ollama model library. [Online]. Available: <https://ollama.com/library/phi4:14b>

- [19] Ollama, “qwen3:30b,” Ollama model library. [Online]. Available: <https://ollama.com/library/qwen3:30b>
- [20] OpenAI, “GPT-4o mini model documentation,” OpenAI Developers. [Online]. Available: <https://developers.openai.com/api/docs/models/gpt-4o-mini>
- [21] OpenAI, “GPT-4.1 mini model documentation,” OpenAI Developers. [Online]. Available: <https://developers.openai.com/api/docs/models/gpt-4.1-mini>